Mastering Relational and Non-Relational Database

CHALLENGE
CLYVO VET

FIAP

Sumario

Folha de rosto 3

Explicação do projeto ..... 4 - 7

Packages 8

Exportação JSON e MongoDB 9

DER 10

MER 11

Folha de rosto

Aluno: Henrique Teixeira Cesar / RM: 563088

Aluno: Lucas Rafael Solimene / RM: 565194

Aluno: Samyr Couto Oliveira / RM: 565562

Explicação projeto

Descrição do Projeto - Sistema de Gestão Veterinária

Este projeto consiste no desenvolvimento de um banco de dados completo para gerenciamento de uma solução veterinária, utilizando Oracle SQL e PL/SQL. O sistema foi projetado para armazenar e relacionar informações de usuários, responsáveis, veterinários, clínicas, pets, consultas, vacinação, lembretes e monitoramento por sensores, mantendo integridade, rastreabilidade e suporte a operações de auditoria.

Modelagem do Banco de Dados

O banco foi estruturado com base em um modelo relacional. As principais entidades da versão atual são:

- Usuário: concentra os dados de autenticação e identificação de responsáveis e veterinários.
- Responsável: perfil associado ao usuário que possui os pets cadastrados.
- Veterinário: profissional associado a um usuário e a uma clínica, identificado por CRV.
- Clínica: local de atendimento ao qual os veterinários e consultas estão vinculados.
- Pet: animal cadastrado, pertencente a um responsável.
- Consulta: atendimento relacionado a pet, veterinário e clínica, com data, motivo, observação e status.
- Vacina e Aplicação de Vacina: catálogo e histórico de aplicações realizadas nos pets.
- Lembrete: compromissos e cuidados futuros vinculados ao pet.
- Sensor e Leitura: estrutura de monitoramento dos pets, incluindo temperatura e atividade.
- Log de Erros e Auditoria: rastreiam falhas de procedimentos e alterações realizadas em consultas.

Relacionamentos

- Um usuário pode representar um responsável ou um veterinário.
- Um responsável pode possuir vários pets; cada pet pertence a um responsável.
- Um veterinário pertence a uma clínica e pode realizar várias consultas e aplicações de vacina.
- Um pet pode possuir várias consultas, aplicações de vacina, lembretes e sensores.
- Cada sensor pertence a um pet e pode registrar várias leituras.
- Chaves primárias, estrangeiras, UNIQUE e CHECK garantem integridade e consistência dos dados.

Functions e Tratamento de Erros

Functions

Foram implementadas duas funções PL/SQL para validação e consulta de dados:

- fn_consulta_json: recebe o ID de uma consulta, realiza JOIN entre Consulta, Pet, Veterinário, Usuário e Clínica e retorna um JSON contendo pet, veterinário, clínica, data, motivo e status.
- fn_validar_cpf: valida o CPF informado, calculando os dois dígitos verificadores e retornando se o documento é válido ou inválido.

Tratamento de Exceções

As functions e procedures utilizam blocos EXCEPTION para tratar situações esperadas e erros inesperados. Entre os casos tratados estão:

- ID nulo, zero ou negativo.
- Consulta ou pet inexistente.
- CPF não informado, com tamanho inválido ou contendo caracteres não numéricos.
- Ausência de leituras, quantidade insuficiente de dados e valores negativos de atividade.
- Erros inesperados capturados por SQLCODE e SQLERRM.

Tabela de Log

A tabela Log_erro armazena as falhas capturadas durante a execução das rotinas PL/SQL. Ela registra:

- Nome da procedure.
- Usuário do banco.
- Data do erro.
- Código do erro.
- Mensagem do erro.

Esse mecanismo permite identificar falhas de execução e manter rastreabilidade durante os testes do sistema.

Procedures e Relatórios

Procedures

Foram implementadas duas procedures principais, integradas às functions e às tabelas de monitoramento:

- pr_consulta_pet_json: recebe o ID de um pet, valida sua existência e verifica se existem consultas cadastradas. Em seguida percorre as consultas do pet e imprime cada registro utilizando fn_consulta_json.
- pr_relatorio_leituras: gera um relatório das leituras de ATIVIDADE dos pets, agrupando os valores por espécie e por pet.

Relatório de Leituras

O relatório de leituras utiliza JOIN entre Leitura, Sensor e Pet, além de estruturas de repetição e variáveis acumuladoras. Durante a execução são apresentados:

- Soma das leituras de atividade de cada pet.
- Subtotal de atividade por espécie.
- Total geral das leituras processadas.
- Ordenação por espécie e nome do pet.

Validações do Relatório

- Nenhuma leitura de atividade disponível.
- Quantidade inferior ao mínimo de cinco leituras.
- Existência de leitura de atividade com valor negativo.

Monitoramento por Sensores

A modelagem inclui sensores vinculados diretamente aos pets. Na carga de dados foram utilizados sensores de TEMPERATURA e ATIVIDADE, e cada leitura registra data, valor e o sensor responsável. Essa estrutura permite integrar o banco ao fluxo IoT do projeto e manter histórico dos dados coletados.

Auditoria e Testes

Trigger de Auditoria

Foi implementada a trigger trg_auditoria_consulta, executada após operações de INSERT, UPDATE ou DELETE na tabela Consulta.

- INSERT: registra os novos dados da consulta.
- UPDATE: registra os valores anteriores e os novos valores.
- DELETE: registra os valores existentes antes da exclusão.
- A operação, usuário, data e tabela afetada são armazenados em Auditoria.

Auditoria

A tabela Auditoria mantém o histórico das alterações realizadas nas consultas. Os campos valor_antigo e valor_novo permitem comparar o estado dos dados antes e depois de uma modificação.

Testes Realizados

Foram preparados testes para validar as rotinas PL/SQL e os mecanismos de rastreabilidade do banco:

- Consulta válida convertida para JSON.
- IDs inválidos e consultas inexistentes.
- CPFs válidos, CPF com dígitos verificadores incorretos, tamanho inválido, caracteres não numéricos e valor nulo.
- Execução normal da procedure de consultas por pet e casos de pet inexistente.
- Execução do relatório de leituras e teste controlado com valor negativo utilizando SAVEPOINT e ROLLBACK.
- INSERT, UPDATE e DELETE de consulta para confirmar o funcionamento da trigger de auditoria.
- Consulta do Log_erro antes e depois de gerar um erro tratado, confirmando o registro da ocorrência.

Os testes abrangem cenários válidos e inválidos e demonstram o funcionamento das validações, exceções, auditoria e log de erros da versão atual do banco.

Packages

Organização em Packages

Na Sprint 4, as functions, procedures e a lógica da trigger foram organizadas em packages, agrupando as rotinas por assunto. Cada package possui uma especificação, que declara as rotinas disponíveis, e um corpo, que contém a implementação.

- pkg_validacao: contém a function fn_validar_cpf.
- pkg_consulta: contém a function fn_consulta_json e a procedure pr_consulta_pet_json. A procedure utiliza a function da própria package para montar o JSON de cada consulta.
- pkg_monitoramento: contém a procedure pr_relatorio_leituras.
- pkg_auditoria: contém a procedure pr_registrar_auditoria, responsável por gravar os registros na tabela Auditoria.
- pkg_exportacao: contém a function fn_pet_json e a procedure pr_exportar_pets, que geram o dataset JSON importado no MongoDB.

Trigger de Auditoria

O Oracle não permite declarar triggers dentro de packages. Por isso, a gravação da auditoria foi movida para a procedure pkg_auditoria.pr_registrar_auditoria, e a trigger trg_auditoria_consulta passou a apenas identificar a operação e chamar a procedure, informando a operação, o valor antigo, o valor novo e a tabela afetada. Assim, o INSERT na tabela Auditoria fica escrito uma única vez e pode ser reaproveitado para auditar outras tabelas.

Vantagens da Organização

- Agrupamento lógico das rotinas por assunto.
- Reutilização de código entre rotinas da mesma package.
- Quem utiliza a package conhece apenas a especificação; a implementação pode ser alterada sem afetar quem chama.
- As rotinas são chamadas com o nome da package, por exemplo pkg_validacao.fn_validar_cpf.

Correções em Relação à Sprint 3

- Log_erro: a coluna id_log_erro passou a usar GENERATED BY DEFAULT AS IDENTITY. Antes, os INSERT de log não informavam o id e a gravação falhava (ORA-01400).
- WHEN OTHERS: SQLCODE e SQLERRM eram usados diretamente no INSERT, o que o Oracle não permite (ORA-00984) e deixava as procedures com status INVALID. Os valores passaram a ser guardados nas variáveis v_codigo e v_mensagem antes do INSERT.
- Usuario: a restrição ck_usuario_tipo passou a aceitar o perfil ADMIN, utilizado pela aplicação Java.

Testes das Packages

- Os testes da Sprint 3 passaram a chamar as rotinas pelo nome da package.
- Exceções da pkg_auditoria: operação inválida e tabela não informada.
- Exceções da pkg_exportacao: ID de pet inválido e pet inexistente.
- Verificação do status VALID das cinco packages e da trigger.

Exportação JSON e MongoDB

Geração do JSON

A procedure pkg_exportacao.pr_exportar_pets percorre todos os pets e gera um array JSON com um documento para cada pet, utilizando a function fn_pet_json. A conversão é feita manualmente com concatenação de texto, sem funções automáticas do Oracle. O resultado foi salvo no arquivo dataset_pets.json.

Estrutura do Documento

No Oracle, os dados de um pet ficam distribuídos em várias tabelas ligadas por chaves estrangeiras. No MongoDB, cada pet se torna um único documento, com os dados relacionados embutidos:

- responsavel: objeto com nome e CPF do responsável.
- consultas: lista com data, motivo, status, veterinário e clínica.
- vacinas: lista com a vacina, a data de aplicação e a dose.
- lembretes: lista com título, data e status.

Dessa forma, todo o histórico do pet é obtido em uma única consulta, sem JOIN entre tabelas.

Estrutura no MongoDB

- Banco clyvo_vet e coleção pets.
- Validação com $jsonSchema: campos obrigatórios, tipos de dados e valores aceitos para o campo sexo, equivalentes às restrições NOT NULL e CHECK do Oracle.
- Índices nos campos especie, responsavel.cpf e lembretes.status, utilizados nas consultas mais frequentes.
- Importação do arquivo dataset_pets.json para a coleção pets.

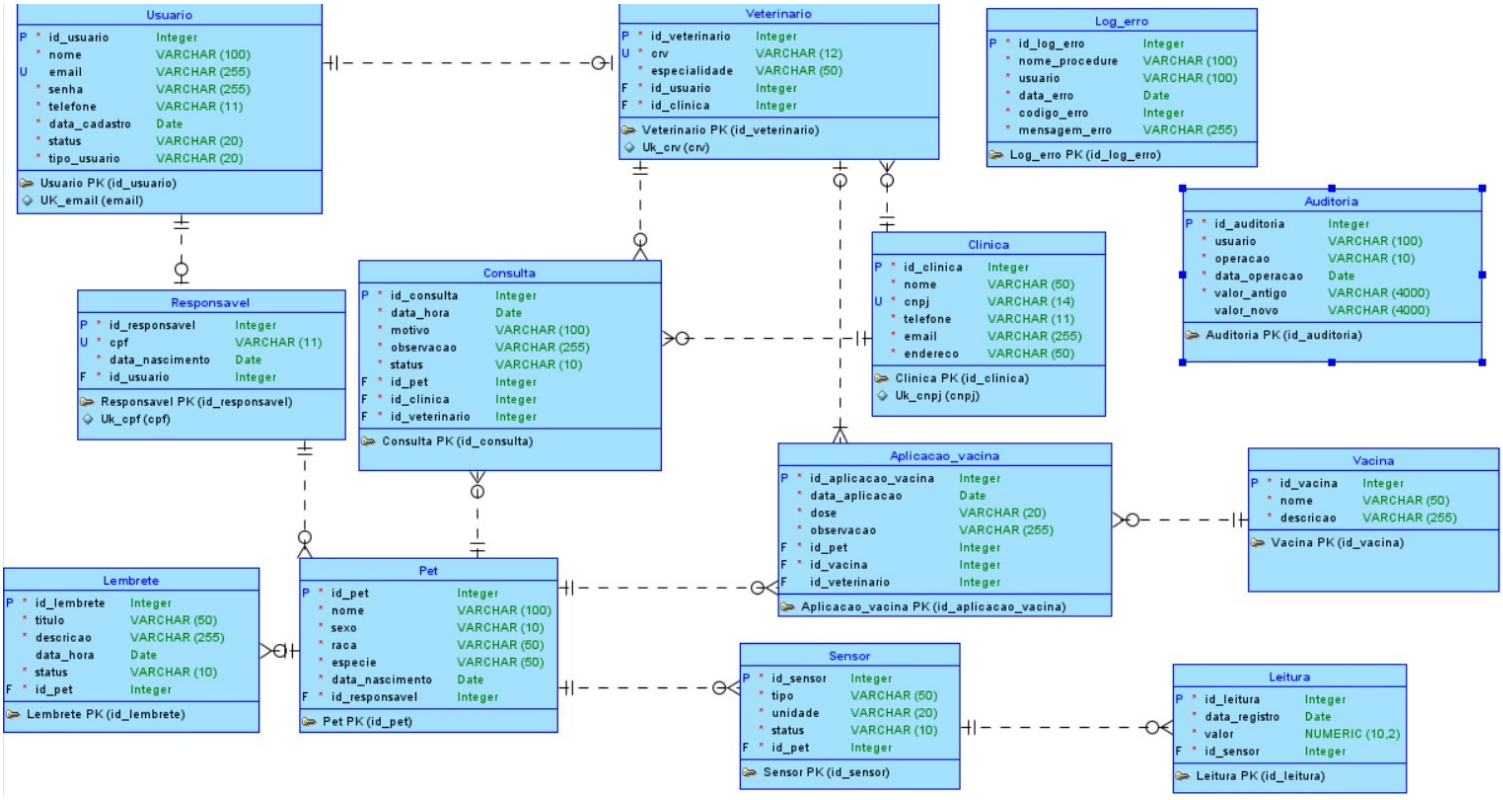
Consultas Realizadas

- Prontuário completo de um pet em uma única leitura (findOne).
- Filtro de pets por espécie.
- Busca dentro de objeto: pets de um responsável pelo CPF.
- Busca dentro de lista: pets com lembrete pendente.
- Contagem de pets por espécie e agrupamento com aggregate ($group).
- Tentativa de inserir um documento sem os campos obrigatórios, recusada pela validação.

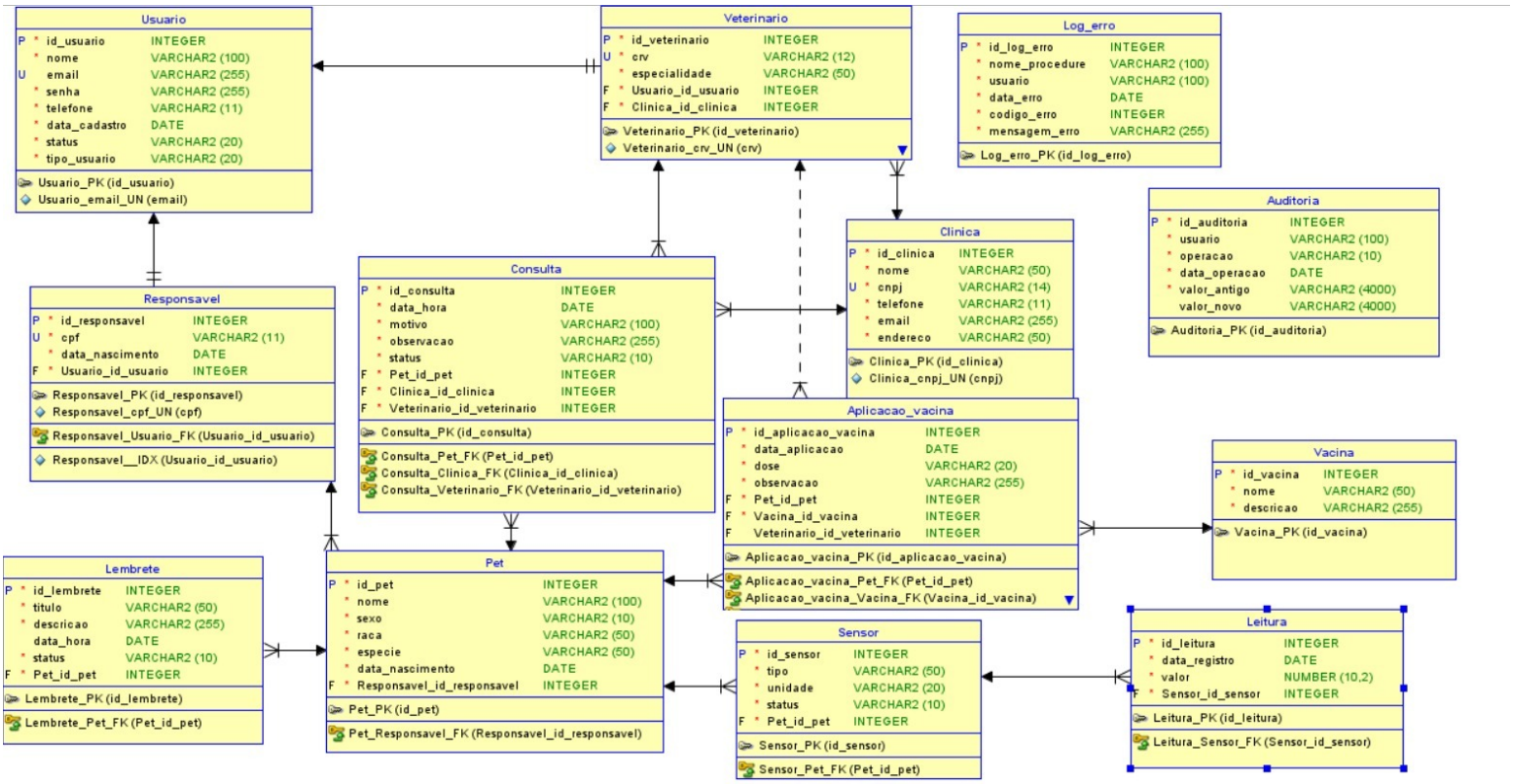
Arquivos

- dataset_pets.json: dataset gerado a partir do banco relacional.
- mongodb/01_criar_estrutura.js: criação da coleção com validação.
- mongodb/02_importacao.txt: instruções e comando de importação.
- mongodb/03_indices.js: criação dos índices.
- mongodb/04_consultas.js: consultas de exemplo.

DER


Usuario
P * id_usuario Integer
* nome VARCHAR (100)
U email VARCHAR (255)
* senha VARCHAR (255)
* telefone VARCHAR (11)
* data_cadastro Date
* status VARCHAR (20)
* tipo_usuario VARCHAR (20)
Usuario PK (id_usuario)
UK_email (email)
Veterinario
P * id_veterinario Integer
U * crv VARCHAR (12)
* especialidade VARCHAR (50)
F * id_usuario Integer
F * id_clinica Integer
Veterinario PK (id_veterinario)
Uk_crv (crv)
Log_erro
P * id_log_erro Integer
* nome_procedure VARCHAR (100)
* usuario VARCHAR (100)
* data_erro Date
* codigo_erro Integer
* mensagem_erro VARCHAR (255)
Log_erro PK (id_log_erro)
Auditoria
P * id_auditoria Integer
* usuario VARCHAR (100)
* operacao VARCHAR (10)
* data_operacao Date
* valor_antigo VARCHAR (4000)
valor_novo VARCHAR (4000)
Auditoria PK (id_auditoria)
Clinica
P * id_clinica Integer
* nome VARCHAR (50)
U * cnpj VARCHAR (14)
* telefone VARCHAR (11)
* email VARCHAR (255)
* endereco VARCHAR (50)
Clinica PK (id_clinica)
Uk_cnpj (cnpj)
Consulta
P * id_consulta Integer
* data_hora Date
* motivo VARCHAR (100)
* observacao VARCHAR (255)
* status VARCHAR (10)
F * id_pet Integer
F * id_clinica Integer
F * id_veterinario Integer
Consulta PK (id_consulta)
Responsavel
P * id_responsavel Integer
U * cpf VARCHAR (11)
* data_nascimento Date
F * id_usuario Integer
Responsavel PK (id_responsavel)
Uk_cpf (cpf)
Aplicacao_vacina
P * id_aplicacao_vacina Integer
* data_aplicacao Date
* dose VARCHAR (20)
* observacao VARCHAR (255)
F * id_pet Integer
F * id_vacina Integer
F * id_veterinario Integer
Aplicacao_vacina PK (id_aplicacao_vacina)
Vacina
P * id_vacina Integer
* nome VARCHAR (50)
* descricao VARCHAR (255)
Vacina PK (id_vacina)
Lembrete
P * id_lembrete Integer
* titulo VARCHAR (50)
* descricao VARCHAR (255)
data_hora Date
* status VARCHAR (10)
F * id_pet Integer
Lembrete PK (id_lembrete)
Pet
P * id_pet Integer
* nome VARCHAR (100)
* sexo VARCHAR (10)
* raca VARCHAR (50)
* especie VARCHAR (50)
* data_nascimento Date
F * id_responsavel Integer
Pet PK (id_pet)
Sensor
P * id_sensor Integer
* tipo VARCHAR (50)
* unidade VARCHAR (20)
* status VARCHAR (10)
F * id_pet Integer
Sensor PK (id_sensor)
Leitura
P * id_leitura Integer
* data_registro Date
* valor NUMERIC (10,2)
F * id_sensor Integer
Leitura PK (id_leitura)

MER


Usuario
P * id_usuario INTEGER
* nome VARCHAR2 (100)
U email VARCHAR2 (255)
* senha VARCHAR2 (255)
* telefone VARCHAR2 (11)
* data_cadastro DATE
* status VARCHAR2 (20)
* tipo_usuario VARCHAR2 (20)
Usuario_PK (id_usuario)
Usuario_email_UN (email)
Veterinario
P * id_veterinario INTEGER
U * crv VARCHAR2 (12)
* especialidade VARCHAR2 (50)
F * Usuario_id_usuario INTEGER
F * Clinica_id_clinica INTEGER
Veterinario_PK (id_veterinario)
Veterinario_crv_UN (crv)
Log_erro
P * id_log_erro INTEGER
* nome_procedure VARCHAR2 (100)
* usuario VARCHAR2 (100)
* data_erro DATE
* codigo_erro INTEGER
* mensagem_erro VARCHAR2 (255)
Log_erro_PK (id_log_erro)
Auditoria
P * id_auditoria INTEGER
* usuario VARCHAR2 (100)
* operacao VARCHAR2 (10)
* data_operacao DATE
* valor_antigo VARCHAR2 (4000)
valor_novo VARCHAR2 (4000)
Auditoria_PK (id_auditoria)
Clinica
P * id_clinica INTEGER
* nome VARCHAR2 (50)
U * cnpj VARCHAR2 (14)
* telefone VARCHAR2 (11)
* email VARCHAR2 (255)
* endereco VARCHAR2 (50)
Clinica_PK (id_clinica)
Clinica_cnpj_UN (cnpj)
Consulta
P * id_consulta INTEGER
* data_hora DATE
* motivo VARCHAR2 (100)
* observacao VARCHAR2 (255)
* status VARCHAR2 (10)
F * Pet_id_pet INTEGER
F * Clinica_id_clinica INTEGER
F * Veterinario_id_veterinario INTEGER
Consulta_PK (id_consulta)
Consulta_Pet_FK (Pet_id_pet)
Consulta_Clinica_FK (Clinica_id_clinica)
Consulta_Veterinario_FK (Veterinario_id_veterinario)
Responsavel
P * id_responsavel INTEGER
U * cpf VARCHAR2 (11)
* data_nascimento DATE
F * Usuario_id_usuario INTEGER
Responsavel_PK (id_responsavel)
Responsavel_cpf_UN (cpf)
Responsavel_Usuario_FK (Usuario_id_usuario)
Responsavel__IDX (Usuario_id_usuario)
Aplicacao_vacina
P * id_aplicacao_vacina INTEGER
* data_aplicacao DATE
* dose VARCHAR2 (20)
* observacao VARCHAR2 (255)
F * Pet_id_pet INTEGER
F * Vacina_id_vacina INTEGER
F Veterinario_id_veterinario INTEGER
Aplicacao_vacina_PK (id_aplicacao_vacina)
Aplicacao_vacina_Pet_FK (Pet_id_pet)
Aplicacao_vacina_Vacina_FK (Vacina_id_vacina)
Vacina
P * id_vacina INTEGER
* nome VARCHAR2 (50)
* descricao VARCHAR2 (255)
Vacina_PK (id_vacina)
Lembrete
P * id_lembrete INTEGER
* titulo VARCHAR2 (50)
* descricao VARCHAR2 (255)
data_hora DATE
* status VARCHAR2 (10)
F * Pet_id_pet INTEGER
Lembrete_PK (id_lembrete)
Lembrete_Pet_FK (Pet_id_pet)
Pet
P * id_pet INTEGER
* nome VARCHAR2 (100)
* sexo VARCHAR2 (10)
* raca VARCHAR2 (50)
* especie VARCHAR2 (50)
* data_nascimento DATE
F * Responsavel_id_responsavel INTEGER
Pet_PK (id_pet)
Pet_Responsavel_FK (Responsavel_id_responsavel)
Sensor
P * id_sensor INTEGER
* tipo VARCHAR2 (50)
* unidade VARCHAR2 (20)
* status VARCHAR2 (10)
F * Pet_id_pet INTEGER
Sensor_PK (id_sensor)
Sensor_Pet_FK (Pet_id_pet)
Leitura
P * id_leitura INTEGER
* data_registro DATE
* valor NUMBER (10,2)
F * Sensor_id_sensor INTEGER
Leitura_PK (id_leitura)
Leitura_Sensor_FK (Sensor_id_sensor)